UNIVERSITÉ DE RENNES 1

ECOLE SUPERIEURE D'INGENIEURS DE RENNES

TP Architecture logiciel

ESIR-AS-A-SERVICE : ARCHITECTURE ET DEPLOIEMENT D'UNE INFRASTRUCTURE MICRO-SERVICES CONTENEURISEE

Réalisé par :

BEHI Kouakou Ange Dominique

KOMBO Madou Hope Othniel

Encadreur pédagogique :

M. WALTER RUDAMETKIN

Classe : SI ESIR 2

Année académique : 2025-2026

SOMMAIRE

INTRODUCTION 3
I. PREREQUIS 4
II. CHOIX ET DESCRIPTION DES SERVICES 4
A. Tableau récapitulatif des services et de leur statut 4
B. Services de base 5
III. ARCHITECTURE GLOBALE 15
A. Vue d'ensemble de l'architecture 15
B. Schéma d'architecture 15
C. Reverse proxy et routage 16
IV. DEPLOIEMENT ET AUTOMATISATION 23
A. Structure du dépôt GitLab 23
B. Processus de déploiement 24
V. PERFORMANCE, MONITORING ET TESTS DE CHARGE 27
A. Outils de monitoring mis en place 27
B. Outil de load-testing 30
C. Load Balancing 31
CONCLUSION 34

INTRODUCTION

Dans le cadre du cours d'Architecture Logicielle, un projet de mise en pratique a été réalisé afin d'appliquer concrètement les notions abordées en théorie. L'objectif est de transformer les concepts de microservices, conteneurisation, reverse proxy, sécurité et monitoring en une infrastructure fonctionnelle et cohérente, déployée sur une machine virtuelle fournie par l'ISTIC. Le but principal est de concevoir et déployer une architecture respectant des exigences de scalabilité, de persistance des données, de sécurité et d'automatisation, au service de la startup ESIR-as-a-Service. Cette infrastructure s'appuie sur des services librement choisis, conteneurisés et orchestrés, afin de fournir une base solide, maintenable et prête à évoluer avec les besoins de l'entreprise.

I. PREREQUIS

Pour la réalisation de ce projet il est primordial d'avoir Docker installé sur la machine. Docker est une plateforme de conteneurisation qui permet d'empaqueter une application avec toutes ses dépendances (système d'exploitation léger, bibliothèques, runtime) dans un conteneur isolé. Contrairement aux machines virtuelles traditionnelles, les conteneurs partagent le noyau du système hôte, ce qui les rend beaucoup plus légers et rapides à démarrer.



Pour ce projet, **il est impératif d'utiliser au minimum Docker Compose version 2** car les scripts d'automatisation Ansible mis en place s'appuient sur les nouvelles fonctionnalités et la syntaxe introduites dans cette version.

Les étapes de l'installation de docker se trouve sur le site officiel au lien suivant : https://docs.docker.com/engine/install/

II. CHOIX ET DESCRIPTION DES SERVICES

A. Tableau récapitulatif des services et de leur statut

Le tableau ci-dessous synthétise l'ensemble des services déployés dans l'infrastructure.

Service	Catégorie	Statut			
		déployé	Configuré et fonctionnel	sécurisé	Load-testé
Trilium Notes	Note-taking	✅	✅	✅	✅
Syncthing	File synchronization	✅	✅	✅	✅
Gitea	Source code management	✅	✅	✅	✅
Mattermost	Instant messaging/chat:	✅	✅	✅	✅
WordPress	Content management	✅	✅	✅	✅
Nextcloud Memories	Photo management	✅	✅	✅	✅
Vaultwarden	Password manager	✅	✅	✅	✅
Wiki.js	Service supplémentaire	✅	✅	✅	✅
Monitoring (Prom + Grafana + exporters)	Optionnel (monitoring)	✅	✅	✅	✅

Pour la sécurité, tous les services sont protégés par authentification et ne sont accessibles qu'aux utilisateurs disposant d'un compte. Les inscriptions sont désactivées publiquement : seul l'administrateur, premier compte créé sur chaque service, peut ajouter de nouveaux utilisateurs, ce qui empêche tout tiers de s'enregistrer librement.

B. Services de base

Pour ce projet, les outils ont été choisis avec l'objectif d'avoir un stack cohérent, simple à maintenir et réellement utilisable en contexte d'entreprise. **Note sur les fichiers de configuration** : Les fichiers Docker Compose et NGINX sont volumineux et répétitifs. Pour des raisons de lisibilité, une description par service a été privilégiée. Les configurations complètes sont disponibles sur le dépôt GitLab ESIR-as-a-Service-2025.

1. Service de prise de notes (Trilium Notes)

- **Présentation et justification du choix**



Pour le service de prise de notes, nous avons choisi **Trilium Notes**, qui offre une interface web multi-utilisateur, une organisation hiérarchique des notes, la recherche pleine-texte et une gestion avancée des balises, idéale pour centraliser la documentation technique de l'infrastructure. Par rapport à des solutions comme Joplin ou Logseq, Trilium est plus adapté à un usage d'équipe auto-hébergé : il se déploie facilement en conteneur, se consulte depuis un navigateur par tous les membres et s'intègre naturellement dans notre architecture de services.

- **Configuration**

Docker compose :

- Service trilium basé sur l'image **zadam/trilium:latest**.
- Redémarrage automatique configuré avec restart: unless-stopped.
- Connecté au réseau Docker esir-net (driver bridge).
- Port interne **8080** exposé sur le port **8180** de la VM (${TRILIUM_HTTP_PORT:-8180}:8080).
- Volume **./data:/home/node/trilium-data** pour la persistance des notes et de la configuration.
- Variable d'environnement **APP_BASE_URL=http://triliumnotes.${DOMAIN_BASE}** pour déclarer l'URL publique du service.

Reverse proxy NGINX :

- Bloc server à l'écoute sur le port **80**.
- Nom de domaine : **triliumnotes.tpdocker.istic.univ-rennes1.fr**.
- Authentification basique activée avec **auth_basic** et fichier **/etc/nginx/.htpasswd**.
- Proxy des requêtes vers **http://localhost:8180**.
- En-têtes Host, X-Real-IP, X-Forwarded-For et X-Forwarded-Proto renseignés pour transmettre à Trilium l'hôte, l'adresse IP cliente et le protocole d'origine.

2. Service de synchronisation de fichiers (Syncthing)

- **Présentation et justification du choix**



Syncthing a été choisi comme service de synchronisation de fichiers car il fournit une synchronisation continue, décentralisée et chiffrée entre plusieurs machines, sans dépendre d'un serveur central unique. Il fonctionne bien dans un contexte d'équipe technique : chaque poste ou serveur peut participer au cluster, les

données restent sous le contrôle de l'entreprise et la configuration reste relativement simple via une interface web.

Par rapport à Nextcloud, Syncthing est plus léger, consomme moins de ressources sur la VM et se concentre exclusivement sur la synchronisation de fichiers, ce qui correspond mieux à notre besoin principal de réplication fiable plutôt qu'à une suite « cloud » complète (calendriers, contacts, suite bureautique, etc.). Par rapport à Seafile, Syncthing évite la complexité d'une base de données et d'un backend plus lourd : il se déploie très facilement en conteneur, ne nécessite pas de composants supplémentaires et reste très robuste pour la synchronisation point-à-point dans une petite infrastructure auto-hébergée.

- **Configuration**

Pour Syncthing, la configuration s'articule autour d'un conteneur dédié et d'un reverse proxy NGINX très proche de celui de Trilium.

Docker compose :

- Service syncthing basé sur l'image linuxserver/syncthing:latest, connecté au réseau esir-net avec redémarrage automatique unless-stopped.
- Variables d'environnement PUID=1000 et PGID=1000 pour que les fichiers créés dans les volumes appartiennent à l'utilisateur standard de la VM, ce qui simplifie les droits sur le système de fichiers.
- Ports exposés :
    - Interface web d'administration sur le port interne 8384, publié sur ${SYNCTHING_HTTP_PORT:-8384}.
    - Port TCP de synchronisation des données 22000 (${SYNCTHING_SYNC_TCP_PORT:-22000}) pour les échanges entre nœuds Syncthing.
    - Port UDP de découverte 21027/udp (${SYNCTHING_DISCOVERY_UDP_PORT:-21027}) pour la détection automatique des pairs.
- Volumes : ./config:/var/syncthing/config pour la configuration et ./data:/var/syncthing/data pour les dossiers synchronisés, assurant la persistance des réglages et des données.

Reverse proxy NGINX :

- Bloc server très similaire à celui de Trilium, à l'écoute sur le port 80 avec authentification basique via .htpasswd.
- Différence principale : le nom de domaine utilisé est syncthing.tpdocker.istic.univ-rennes1.fr et le proxy redirige vers http://localhost:8384, correspondant à l'interface web d'administration du conteneur Syncthing.

3. Gestionnaire de code source (Gitea)

❖ **Présentation et justification du choix**



Pour la gestion du code source, nous avons choisi **Gitea** car il offre une forge Git complète (dépôts, issues, wiki, revue de code) tout en restant très légère, ce qui convient bien aux ressources limitées de notre VM.

Par rapport à **GitLab**, Gitea consomme **beaucoup moins de RAM** et d'espace disque pour un usage similaire dans un petit projet. Par rapport à **Fossil**, Gitea reste plus proche des outils utilisés en entreprise (Git et workflows classiques), ce qui le rend plus naturel pour les développeurs tout en gardant une empreinte système minimale.

❖ **Configuration**

Docker compose :

➢ **Base de données PostgreSQL**

- Service db basé sur l'image postgres:15-alpine, connecté au réseau esir-net avec redémarrage unless-stopped.
- Variables d'environnement : POSTGRES_USER=gitea, POSTGRES_PASSWORD=gitea_password, POSTGRES_DB=giteadb.
- Volume ./db:/var/lib/postgresql/data pour persister les dépôts et la configuration de Gitea côté base de données.

➢ **Service Gitea**

- Service gitea basé sur l'image gitea/gitea:latest, dépendant du service db et relié au même réseau esir-net.
- Ports exposés : interface web sur le port interne 3000 publié en 8083:3000, accès SSH Git sur 2222:22.
- Volume ./data:/data pour stocker de façon persistante les dépôts Git, les avatars, logs et fichiers de configuration.
- Variables USER_UID=1000 et USER_GID=1000 pour aligner les droits fichiers avec l'utilisateur de la VM.
- Sécurisation de l'instance via :
- GITEA__service__REQUIRE_SIGNIN_VIEW=true (nécessité d'être connecté pour voir les dépôts).
- GITEA__service__DISABLE_REGISTRATION=true (inscriptions publiques désactivées, seuls les comptes créés par l'admin sont autorisés).
- Configuration de la base de données : DB_TYPE=postgres, DB_HOST=db:5432, DB_NAME=giteadb, DB_USER=gitea, DB_PASSWD=gitea_password.

- Configuration
  HTTP : GITEA__server__DOMAIN=gitea.${DOMAIN_BASE}, GITEA__server__ROOT_URL=http://gitea.${DOMAIN_BASE}/, GITEA__server__HTTP_PORT=3000.

Reverse proxy NGINX :

- Bloc server écoutant sur le port 80 pour gitea.tpdocker.istic.univ-rennes1.fr.
- Proxy des requêtes vers http://localhost:8083, correspondant au port HTTP publié par le conteneur Gitea.
- En-têtes Host, X-Real-IP, X-Forwarded-For et X-Forwarded-Proto définis pour transmettre correctement l'hôte et l'adresse du client à Gitea.
- Directive client_max_body_size 500M pour permettre le push de dépôts de taille significative.
- Pas d'authentification basique devant Gitea afin de ne pas casser les opérations Git à distance (clone, pull, push) via HTTP/SSH.

4. Messagerie instantanée / chat

- **Présentation et justification du choix**

Nous avons choisi **Mattermost** car il offre une messagerie d'équipe complète, simple à déployer et adaptée à une petite infrastructure. Par rapport à **Rocket.Chat**, Mattermost est plus léger et plus stable pour un auto-hébergement minimal. Face à **Matrix**, dont l'architecture fédérée est plus complexe et gourmande en ressources, Mattermost reste plus facile à administrer tout en couvrant les besoins essentiels de communication interne.

- **Configuration**

Docker compose :

- Service mattermost déployé à partir des fichiers docker-compose.yml et docker-compose.without-nginx.yml complémentaires, avec une politique de redémarrage unless-stopped et exposition de l'application sur le port 8065 de la VM (variable MATTERMOST_APP_PORT).
- Volumes Mattermost définis via le fichier .env (MATTERMOST_DATA_PATH, MATTERMOST_LOGS_PATH, MATTERMOST_PLUGINS_PATH, etc.) pour stocker de façon persistante les données, les journaux et les plugins de l'application dans le répertoire ./volumes/app/mattermost.
- Base de données PostgreSQL configurée par les variables POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB et utilisée dans MM_SQLSETTINGS_DATASOURCE pour construire l'URL postgres://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432/${POSTGRES_DB}?sslmode=disable&connect_timeout=10.

- Paramètres applicatifs Mattermost définis dans .env, notamment l'image mattermost-enterprise-edition:10.11.5, le fuseau horaire TZ, l'URL publique du site MM_SERVICESETTINGS_SITEURL=https://${DOMAIN} et le port applicatif interne APP_PORT=8065.
- Ports des appels audio/vidéo exposés via CALLS_PORT en TCP et UDP, afin de permettre les fonctionnalités de « Calls » de Mattermost.

Reverse proxy NGINX :

- Bloc server très similaire à celui utilisé pour Trilium : écoute sur le port 80, authentification basique avec auth_basic et fichier /etc/nginx/.htpasswd.
- Différences principales : le nom de domaine est mattermost.tpdocker.istic.univ-rennes1.fr et le proxy_pass pointe vers http://localhost:8065, c'est-à-dire le port publié par le conteneur Mattermost via docker-compose.without-nginx.yml.

5. Gestion de contenu (WordPress)

❖ **Présentation et justification du choix**



Pour le site web de la startup, nous avons choisi WordPress, qui offre une interface d'édition très accessible, un grand écosystème de thèmes/plugins et la possibilité de créer rapidement une page d'accueil complète avec blog, documentation et liens vers nos autres services.

Par rapport à Ghost, WordPress est plus polyvalent : au-delà du simple blog, il permet facilement d'ajouter des pages, formulaires, intégrations et fonctionnalités variées sans développement spécifique, ce qui est pratique pour un projet pédagogique amené à évoluer.

Face à Hugo, qui génère un site statique très performant mais nécessite un pipeline de génération et plus de manipulation de fichiers, WordPress reste plus simple à administrer pour l'équipe : tout se fait via le navigateur, sans recompilation, ce qui réduit la charge opérationnelle tout en restant suffisamment performant pour notre usage.

❖ **Configuration**

Docker compose :

- Service wordpress basé sur l'image wordpress:6.4-apache.
- Redémarrage automatique configuré avec restart: always.
- Dépend du service mariadb basé sur l'image mariadb:10.6 avec restart: always.
- Variables d'environnement MYSQL_DATABASE=wordpress, MYSQL_USER=wp, MYSQL_PASSWORD=wppassword, MYSQL_ROOT_PASSWORD=rootpassword pour initialiser la base.
- Port interne 80 exposé sur le port 8081 de la VM (8081:80).

- Volume wordpress_app_data:/var/www/html pour la persistance du code, des thèmes, des plugins et des fichiers uploadés.
- Variables d'environnement WORDPRESS_DB_HOST=mariadb:3306, WORDPRESS_DB_USER=wp , WORDPRESS_DB_PASSWORD=wppassword, WORDPRESS_DB_NAME=wordpress po ur configurer la connexion à la base de données.

Reverse proxy NGINX :

- Bloc server très similaire à celui de Trilium, à l'écoute sur le port 80 avec authentification basique via auth_basic et fichier /etc/nginx/.htpasswd.
- Différence principale : le nom de domaine utilisé est wordpress.tpdocker.istic.univ-rennes1.fr et le proxy redirige vers http://localhost:8081, correspondant au port publié par le conteneur WordPress.

6. Gestion des photos (Nextcloud memorie)

❖ **Présentation et justification du choix**

Pour la gestion et le partage des photos, nous avons choisi **Nextcloud Memories**, qui offre une interface moderne (timeline, albums, recherche) tout en restant relativement simple à déployer dans une unique instance Nextcloud dédiée au projet. Par rapport à **Immich**, Nextcloud Memories est moins exigeant en ressources et propose aussi des fonctions de stockage/partage de fichiers plus générales, ce qui le rend plus polyvalent pour notre petite infrastructure. Face à **Ente**, qui reste très orienté service cloud avec une offre hébergée, Nextcloud Memories nous donne un contrôle complet sur les données dans un environnement entièrement auto-hébergé.

❖ **Configuration**

Docker compose :

➢ Conteneur Nextcloud (app)
- Service app basé sur l'image nextcloud:stable.
- Redémarrage automatique configuré avec restart: always.
- Dépend du service db pour la base de données.
- Port interne 80 exposé sur le port 8080 de la VM.
- Volume nextcloud:/var/www/html pour la persistance des fichiers de l'application, des extensions et des données utilisateur.
- Variables d'environnement MYSQL_PASSWORD=userpass123, MYSQL_DATABASE=nextcloud, MYSQL_USER=nextcloud, MYSQL_HOST=db pour configurer la connexion à la base de données.

➢ Base de données MariaDB
- Service db basé sur l'image mariadb:10.6.

- Redémarrage automatique configuré avec restart: always.
- Commande personnalisée --transaction-isolation=READ-COMMITTED --log-bin=binlog --binlog-format=ROW recommandée pour Nextcloud afin de garantir la cohérence des transactions.
- Variables d'environnement MYSQL_ROOT_PASSWORD=rootpass123, MYSQL_PASSWORD=userpass123, MYSQL_DATABASE=nextcloud, MYSQL_USER=nextcloud pour initialiser le schéma et l'utilisateur applicatif.
- Volume db:/var/lib/mysql pour la persistance des données SQL.
- Port interne 3306 exposé sur 3307 de la VM.

Reverse proxy NGINX :

- Bloc server très similaire à celui de Trilium, à l'écoute sur le port 80.
- Différence principale : le nom de domaine utilisé est nextcloud.tpdocker.istic.univ-rennes1.fr et le proxy redirige vers http://localhost:8080, correspondant au port HTTP publié par le conteneur Nextcloud.

7. Gestionnaire de mots de passe partagé (Vaultwarden)

❖ **Présentation et justification du choix**

Vaultwarden a été choisi car il fournit un serveur compatible Bitwarden très léger, simple à déployer en conteneur et bien adapté aux ressources limitées de notre VM.

Par rapport au serveur Bitwarden officiel, il consomme moins de RAM et de stockage, tout en restant utilisable avec les clients Bitwarden classiques. Contrairement à KeePassXC, basé sur un fichier local à synchroniser, Vaultwarden offre une expérience centralisée multi-appareils plus pratique pour partager des coffres entre membres de l'équipe.

❖ **Configuration**

Docker compose :

- Service vaultwarden basé sur l'image vaultwarden/server:latest.
- Redémarrage automatique configuré avec restart: unless-stopped.
- Port interne 80 exposé sur le port 4044 de la VM (4044:80).
- Volume ./vw-data:/data pour la persistance des coffres chiffrés et de la configuration du serveur.
- Variables d'environnement de sécurité : ADMIN_TOKEN="mydockerpass404" pour protéger l'interface d'administration, SIGNUPS_ALLOWED="false" pour interdire les inscriptions publiques et INVITATIONS_ALLOWED="true" pour permettre uniquement la création de comptes via invitation.

Reverse proxy NGINX :

- Bloc server très similaire à celui de Trilium, à l'écoute sur le port 80 avec authentification basique via auth_basic et fichier /etc/nginx/.htpasswd.
- Différences principales : le nom de domaine utilisé est vaultwarden.tpdocker.istic.univ-rennes1.fr et le proxy redirige vers http://localhost:4044, correspondant au port HTTP publié par le conteneur Vaultwarden.
- Localisation supplémentaire /notifications/hub configurée pour les WebSockets de Vaultwarden, avec Upgrade et Connection "upgrade" afin de supporter les notifications temps réel dans les clients comme décrit dans l'image ci-dessous.

```
            # WebSocket pour Vaultwarden
location /notifications/hub {
    proxy_pass http://localhost:4044;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_set_header Host $host;
}
```

8. Service supplémentaire choisi (Wiki.js)

❖ **Présentation et justification du choix**

Wiki.js est un wiki moderne orienté documentation, qui permet de créer et organiser des pages au format Markdown avec une interface web agréable et un moteur de recherche intégré. Il supporte la hiérarchie de pages, les historiques de modifications, la gestion fine des permissions et l'intégration de médias ou de blocs de code, ce qui en fait un bon support pour documenter l'architecture, les procédures et les décisions techniques.

Pour une équipe ou une startup, un outil comme Wiki.js est particulièrement intéressant car il centralise la connaissance dans un espace unique, versionné et collaboratif, au lieu de laisser les informations éparpillées dans des documents ou des conversations de chat. Cela facilite l'onboarding des nouveaux membres, la transmission des bonnes pratiques et la mise à jour

❖ **Configuration**

Docker compose :

➢ **Base de données PostgreSQL**

- Service db basé sur l'image postgres:15-alpine.
- Redémarrage automatique configuré avec restart: unless-stopped et logs désactivés (logging.driver: none) pour limiter l'occupation disque.
- Variables d'environnement POSTGRES_DB=wiki, POSTGRES_USER=wikijs, POSTGRES_PASSWORD=wikijsrocks pour initialiser la base utilisée par Wiki.js.
- Volume db-data:/var/lib/postgresql/data pour la persistance des données.

➢ **Conteneur Wiki.js**

- Service wiki basé sur l'image ghcr.io/requarks/wiki:2.
- Dépend du service db pour la base de données.
- Redémarrage automatique configuré avec restart: unless-stopped.
- Port interne 3000 exposé sur le port 8082 de la VM (8082:3000).
- Variables d'environnement :
- DB_TYPE=postgres, DB_HOST=db, DB_PORT=5432, DB_USER=wikijs, DB_PASS=wikijsrocks, DB_NAME=wiki pour la connexion à PostgreSQL.
- APP_URL="http://wiki.tpdocker.istic.univ-rennes1.fr/" pour définir l'URL publique de l'application, utilisée par l'interface et les liens générés.

Reverse proxy NGINX :

- Bloc server à l'écoute sur le port 80 pour le domaine wiki.tpdocker.istic.univ-rennes1.fr.
- Authentification basique activée globalement avec auth_basic et auth_basic_user_file /etc/nginx/.htpasswd.
- Bloc location ^~ /api/ :
    - auth_basic off pour permettre aux appels API internes de Wiki.js de fonctionner sans authentification HTTP supplémentaire.
    - Proxy des requêtes vers http://localhost:8082 avec support WebSocket (proxy_http_version 1.1, Upgrade, Connection "upgrade").
    - En-têtes Host, X-Real-IP, X-Forwarded-* renseignés pour transmettre correctement les informations de client et de port.
- Bloc location /graphql configuré de la même manière que /api/ (auth désactivée et WebSocket activé) pour les requêtes GraphQL de l'interface.
- Bloc location / :
    - Proxy principal de l'interface web vers http://localhost:8082 avec proxy_http_version 1.1 et support WebSocket.

- Même série d'en-têtes Host et X-Forwarded-* pour assurer le bon fonctionnement de Wiki.js derrière le proxy.

III. ARCHITECTURE GLOBALE

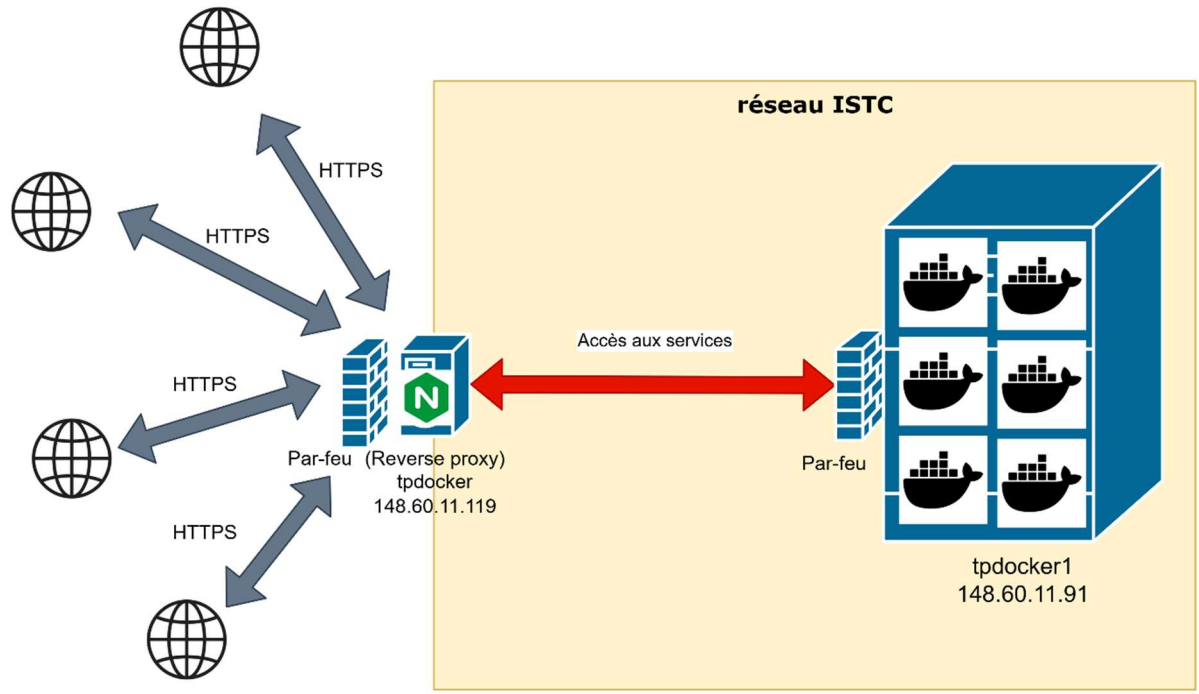
A. Vue d'ensemble de l'architecture

L'architecture repose sur plusieurs micro-services Docker, chacun dans son propre conteneur avec ses volumes de données et, si besoin, sa base de données dédiée.

Les services sont répartis sur **deux VMs** : **tpdocker** est la seule machine **exposée à Internet** et joue le rôle de frontal avec **NGINX**, qui reçoit tout le trafic et le redirige vers les bons services. La VM **tpdocker1**, elle, n'est jamais exposée directement : elle héberge tous les conteneurs applicatifs et n'est accessible que depuis tpdocker sur le réseau interne ISTIC, ce qui **réduit fortement la surface d'attaque** et limite l'impact potentiel d'un compromis du reverse proxy.

B. Schéma d'architecture

Ci-dessous l'architecture du système mis en place :



Sécurité de l'architecture réseau

L'architecture implémente un modèle de sécurité fondé sur la séparation stricte des responsabilités réseau. **Seule la VM tpdocker (148.60.11.119), qui héberge le reverse proxy NGINX, est exposée à Internet** et reçoit directement le trafic HTTP/HTTPS provenant des utilisateurs externes. Cette machine constitue le point d'entrée unique et applique

l'authentification HTTP basic ainsi que les règles de routage centralisées vers les différents services.

La VM tpdocker1 (148.60.11.91), qui héberge tous les services applicatifs, n'est jamais exposée directement sur Internet : elle reste confinée au réseau interne ISTIC et n'écoute pas sur des ports publics accessibles depuis l'extérieur. Les utilisateurs ne peuvent atteindre les services qu'en passant par tpdocker, ce qui garantit que tout le trafic vers les conteneurs est d'abord filtré et routé par NGINX.

Cette approche en deux couches réduit significativement la surface d'attaque : plutôt que d'exposer de nombreux ports applicatifs, seuls les ports 80 et/ou 443 de NGINX sont visibles depuis l'extérieur. Les ports internes des services (par exemple 3000, 3306, 5432, 8080, etc.) restent limités au réseau interne, ce qui diminue les risques liés au scan de ports ou à l'exploitation directe de vulnérabilités sur les services applicatifs.

C. Reverse proxy et routage

1. Choix du reverse proxy et justification

Pour le reverse proxy, nous avons finalement retenu **NGINX** après avoir hésité avec Traefik. Traefik est très adapté pour gérer dynamiquement plusieurs services, mais NGINX offre des performances meilleures et une empreinte mémoire plus faible, ce qui est important sur notre VM limitée à 4 Go de RAM.

Après une première configuration, la création manuelle des fichiers de virtual host pour chaque nom de domaine/sous-domaine s'est révélée moins complexe que prévu. La combinaison « **simplicité de configuration** + **très bonnes performances** » nous a donc convaincus de standardiser tout le routage HTTP/HTTPS sur NGINX pour l'ensemble des services.

2. Configuration générale

La mise en place suit toujours les mêmes étapes générales :

1. **Démarrage ou redémarrage du service Docker**
    - Se placer dans le répertoire contenant le fichier docker-compose.yml du service.
    - Lancer ou relancer les conteneurs :

```
docker compose up -d
```

Vérifier que les conteneurs sont bien en cours d'exécution :

```
docker ps
```

2. **Installation de NGINX sur la VM frontale (une seule fois)**

- Mettre à jour les paquets et installer NGINX (si ce n'est pas déjà fait) :

- `sudo apt update && sudo apt install nginx`

3. **Création du vhost pour un service donné (nomService)**

- Créer un fichier de configuration dans `/etc/nginx/sites-available/nomService.conf`.

- Y définir un bloc server qui pointe vers le port local publié par Docker en remplaçant `<vm>.istic.univ-rennes1.fr` par le nom de domaine correspondant , par exemple :

```
server {
  listen 80;
  server_name nomService.tpdocker.istic.univ-rennes1.fr;

  location / {
    proxy_pass http://localhost:PORT_SERVICE;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
  }
}
```

Pour certains services (par exemple Wiki.js ou Mattermost), ajouter au besoin des location spécifiques (/api/, /graphql/, WebSocket, etc.) ou désactiver auth_basic sur ces chemins lorsque l'application a besoin d'y accéder sans authentification HTTP supplémentaire.

4. Activation d'un vhost

```
sudo ln -s /etc/nginx/sites-available/nomService.conf /etc/nginx/sites-enabled/
```

Pour désactiver ultérieurement un vhost au besoin :
`sudo unlink /etc/nginx/sites-enabled/nomService.conf`

5. Rechargement de NGINX

```
sudo nginx -t

sudo systemctl reload nginx
```

6. Test d'accès depuis l'extérieur

Depuis un navigateur, accéder à l'URL du service: **http://nomService.<vm>.istic.univ-rennes1.fr**

3. Basic authentification avec nginx

La sécurité est renforcée par l'usage systématique d'une authentification HTTP basique sur les services qui n'intègrent pas eux-mêmes de mécanisme de login, par la limitation de l'exposition des ports (seuls Nginx et quelques endpoints nécessaires sont accessibles depuis l'extérieur) et par la séparation des services sur un réseau Docker dédié pour réduire la surface d'attaque. Les étapes sont les suivantes:

1. **Installe le paquet htpasswd si ce n'est pas déjà fait :**

```
sudo apt install apache2-utils
```

2. **Crée le fichier des utilisateurs :**

```
htpasswd -c /etc/nginx/.htpasswd nomUtilisateur
```

3. **Configuration du bloc Nginx**

- Ajouter les directives suivantes dans le bloc server du sous-domaine à protéger, ou dans un location précis :

```
auth_basic "Accès réservé";
auth_basic_user_file /etc/nginx/.htpasswd;
```

- Pour certains endpoints techniques (par exemple /api/ ou /graphql de Wiki.js), désactiver explicitement l'authentification pour permettre aux appels internes de fonctionner. Exemple pour protéger tout un sous-domaine :

```
server {
  listen 80;
  server_name sousdomaine.tondomaine.fr;
  auth_basic "Accès réservé";
  auth_basic_user_file /etc/nginx/.htpasswd;
  location / {
    # autres options proxy habituelles
  }
}
```

4. Passer en HTTPS

Étape 1 – Préparer le répertoire ACME dans /opt (ou le dossier choisi).

Créer l'arborescence :

```
sudo mkdir -p /opt/acme/.well-known/acme-challenge
sudo chown -R kbehi:kbehi /opt/acme
sudo chmod -R 755 /opt/acme
```

Les challenges et certificats Let's Encrypt seront stockés sous /opt/acme.

Étape 2 – Configurer Nginx pour HTTP + ACME

Objectif :

- Port 80 ouvert pour les challenges ACME
- Redirection HTTP → HTTPS pour tout le reste

Fichier /etc/nginx/sites-available/default :

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;
    server_name _;

    # 1) Challenge ACME : pas de redirection ici
    location ^~ /.well-known/acme-challenge/ {
        root /opt/acme;
        try_files $uri =404;
    }
    # 2) Tout le reste : redirection vers HTTPS
    location / {
        return 301 https://$host$request_uri;
    }
}
```

Lancer :

```
sudo nginx -t

sudo systemctl reload nginx
```

Test manuel :

```
echo "ok" | sudo tee /opt/acme/.well-known/acme-challenge/test.txt

curl http://tpdocker.istic.univ-rennes1.fr/.well-known/acme-challenge/test.txt

# → doit afficher "ok"
```

Étape 3 – Obtenir un certificat Let's Encrypt avec acme.sh (multidomaines)

On utilise acme.sh via Docker, avec /opt/acme monté dans le conteneur.

Commande type (adapter la liste des -d à tes sous-domaines) :

```
sudo docker run --rm -it \
 -v "/opt/acme":/acme.sh \
 neilpang/acme.sh --issue --server letsencrypt \
 -d tpdocker.istic.univ-rennes1.fr \
 -d nextcloud.tpdocker.istic.univ-rennes1.fr \
 -d syncthing.tpdocker.istic.univ-rennes1.fr \
 -d xxx.tpdocker.istic.univ-rennes1.fr \
 -w /acme.sh
```

- Dans le conteneur, acme.sh écrit les challenges dans /acme.sh/.well-known/acme-challenge/…
- Sur la VM, cela correspond à /opt/acme/.well-known/acme-challenge/…, ce que Nginx sert déjà.

Si tout se passe bien, le certificat et la clé seront créés dans :

/opt/acme/tpdocker.istic.univ-rennes1.fr_ecc/

├– fullchain.cer

├– tpdocker.istic.univ-rennes1.fr.key

└ ...

Étape 4 – Activer HTTPS pour le site principal (tpdocker dans notre cas)

On garde le server HTTP (port 80) du default pour ACME + redirection, et on ajoute un bloc HTTPS 443 :

```
# HTTP déjà défini comme à l'étape 2
# HTTPS : page d'accueil tpdocker
server {
   listen 443 ssl http2 default_server;
   listen [::]:443 ssl http2 default_server;
   server_name tpdocker.istic.univ-rennes1.fr;
   root /var/www/html/tpdocker;
   index index.html index.htm index.nginx-debian.html;
   ssl_certificate     /opt/acme/tpdocker.istic.univ-rennes1.fr_ecc/fullchain.cer;
   ssl_certificate_key       /opt/acme/tpdocker.istic.univ-rennes1.fr_ecc/tpdocker.istic.univ-rennes1.fr.key;
   ssl_protocols TLSv1.2 TLSv1.3;
   ssl_ciphers HIGH:!aNULL:!MD5;
   location / {
      try_files $uri $uri/ =404;
   }
}
```

Puis :

```
sudo nginx -t
sudo systemctl reload nginx
```

Tester votre nom de domaine en https (https://...) .

Étape 5 – Activer HTTPS pour chaque service (reverse proxy)

Pour chaque service, on modifier le fichier de configuration dédié dans /etc/nginx/sites-available/, par exemple nextcloud.conf :

```
# HTTP : redirection + ACME
server {
   listen 80;
   server_name nextcloud.tpdocker.istic.univ-rennes1.fr;
```

```
    location ^~ /.well-known/acme-challenge/ {
        root /opt/acme;
        try_files $uri =404;
    }
    location / {
        return 301 https://$host$request_uri;
    }
}
# HTTPS : proxy vers le conteneur Docker
server {
    listen 443 ssl http2;
    listen [::]:443 ssl http2;
    server_name wiki.tpdocker.istic.univ-rennes1.fr;
    ssl_certificate     /opt/acme/tpdocker.istic.univ-rennes1.fr_ecc/fullchain.cer;
    ssl_certificate_key       /opt/acme/tpdocker.istic.univ-rennes1.fr_ecc/tpdocker.istic.univ-
rennes1.fr.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    location / {
        proxy_pass http://localhost:PORT_NEXTCLOUD;  # adapter PORT
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

À faire pour chaque service puis rechargez nginx :
sudo nginx -t sudo systemctl reload nginx

IV. DEPLOIEMENT ET AUTOMATISATION

A. Structure du dépôt GitLab

Le dépôt est organisé de façon logique pour que chaque service soit facilement identifiable et maintenable. Chaque dossier correspond à un service ou à un domaine fonctionnel.

Dossiers des services applicatifs
Chacun contient un fichier docker-compose.yml spécifique au service et, si nécessaire, les fichiers de configuration (fichiers nginx pour les services qui ont besoin, volumes, etc.) :

- triliumnottes/ : prise de notes
- syncthing/ : synchronisation de fichiers
- gitea/ : gestion du code source avec PostgreSQL
- mattermost/ : messagerie d'équipe avec PostgreSQL
- wordpress/ : site web avec MariaDB
- nextcloud/ : gestion de fichiers et photos avec MariaDB
- vaultwarden/ : gestionnaire de mots de passe partagé
- wikijs/ : wiki de documentation avec PostgreSQL

Dossiers d'infrastructure et monitoring

- monitoring/ : Grafana, Prometheus, cAdvisor et Node Exporter pour la surveillance et l'analyse des performances de tous les services
- loadtesting/ : k6 pour les tests de charge de l'infrastructure

Dossier reverse proxy

- configs/ : toutes les configurations NGINX (http et https) (blocs server, virtual hosts pour les sous-domaines, règles de proxy, authentification basique)

Dossier d'automatisation

- playbooks_ansible/ : fichiers playbook.yml permettant l'automatisation du déploiement via Ansible, incluant l'initialisation des VMs, l'installation de dépendances et le lancement des services

Fichiers de configuration et déploiement

- .env_example : fichier d'exemple à dupliquer en .env pour configurer globalement tous les services (ports, domaines, noms de bases de données, mots de passe, variables d'environnement communes)
- deploy-all.yml : point d'entrée du déploiement Ansible, qui référence et exécute tous les playbooks du dossier playbooks_ansible/
- inventory.ini : fichier d'inventaire Ansible décrivant les VMs, les groupes d'hôtes et les variables (par exemple les adresses IP de tpdocker et tpdocker1)
- deploy-without-nginx.sh : script de déploiement qui publie chaque service directement sur un port de la VM, accessible en http://IP_VM:PORT sans passer par Nginx.
- deploy-services.sh : script de déploiement qui expose les services via Nginx en utilisant DOMAIN_BASE et les fichiers de configuration de configs/http.
- clean-services.sh : script de nettoyage complet qui arrête et supprime tous les services Docker du projet, détruit le réseau esir-net et désactive / supprime les vhosts Nginx associés.

Documentation

- README.md : instructions de déploiement, description générale des services et accès aux configurations principales

Cette organisation permet de cloner le dépôt sur une machine fraîche, de personnaliser le fichier .env et de lancer l'ensemble de l'infrastructure soit via Ansible soit via le script shell, en garantissant la cohérence des configurations et la facilité de maintenance.

B. Processus de déploiement

1. Déploiement à partir du script shell

Le déploiement via le script shell offre une approche simple et directe pour mettre en place l'infrastructure complète. Bien évidement il faut avoir **docker et docker compose installés** sur la machine. Voici les étapes à suivre :

1. Cloner le dépôt GitLab
Se placer dans le répertoire de travail souhaité et cloner le dépôt du projet :

git clone https://gitlab2.istic.univ-rennes1.fr/esir-as-a-service-2025-docker/esir-as-a-service-2025.git

cd ESIR-as-a-Service-2025

2.Créer le fichier .env personnalisé
À la racine du projet, dupliquer le fichier .env_example en .env :

cp .env_example .env

Éditer le fichier .env pour adapter les configurations selon votre environnement (noms de domaines, ports…).

3.Rendre le script exécutable

Donner les droits d'exécution au script de déploiement :

```
chmod +x script-deploy-services.sh
```

4.Lancer le déploiement

Exécuter le script depuis la racine du projet :

```
./script-deploy-services.sh
```

Le script effectuera automatiquement le lancement de tous les conteneurs via docker compose up -d pour chaque service.

5.Vérifier l'état du déploiement

Une fois le script terminé, vérifier que tous les conteneurs sont en cours d'exécution :

```
docker ps
```

Vous devriez voir l'ensemble des services listés (Trilium, Syncthing, Gitea, Mattermost, WordPress, Nextcloud, Vaultwarden, Wiki.js, monitoring, etc.). En cas d'erreur, consulter les logs du service concerné :

```
docker logs nom_du_conteneur
```

2. Déploiement à partir de ansible (avancée)

Ansible offre une approche plus robuste et répétable pour automatiser le déploiement complet de l'infrastructure sur les VMs. Cette section décrit les prérequis, l'installation et les étapes de déploiement.

1.Prérequis

- Disposer d'un poste de contrôle (machine locale ou serveur) d'où lancer les commandes Ansible.
- Pouvoir accéder en SSH à la machine sur laquelle on veut déployer les services sans mot de passe ou avec authentification par clé.
- Avoir python installé

2.Installation d'Ansible

Sur le poste de contrôle, installer Ansible (sur Debian/Ubuntu) :

```
sudo apt update
```

```
sudo apt install ansible
```

Vérifier l'installation :

```
ansible --version
```

3.Configuration du fichier inventory.ini

À la racine du projet, modifier le fichier inventory.ini pour y indiquer les adresses IP ou noms de domaine de vos VMs, les utilisateurs, et les groupes d'hôtes. Un exemple typique :

```
[vm1]

vm-tpdocker ansible_host=tpdocker1.istic.univ-rennes1.fr ansible_user=makombo ansible_ssh_private_key_file=~/.ssh/id_rsa

[vm2] tpdocker ansible_host=tpdocker.istic.univ-rennes1.fr ansible_user=kbehi
```

4.Vérifier que Ansible peut atteindre les VMs :

```
ansible all -i inventory.ini -m ping
```

5.Préparation du fichier .env

Comme avec le script shell, dupliquer .env_example en .env et adapter les paramètres selon votre environnement :

```
cp .env_example .env
```

6.Lancer le déploiement complet

Une fois les configurations ajustées, lancer le playbook principal depuis la racine du projet :

```
ansible-playbook -i inventory.ini deploy-all.yml
```

Ansible exécutera automatiquement les étapes suivantes sur les VMs respectives :

- Vérification des prérequis : vérifie que Docker et Docker Compose v2 sont installés sur chaque VM. Si l'un des deux est absent, le déploiement s'arrête avec un message d'erreur clair pour corriger le problème.
- Création des répertoires de projet : pour chaque service, Ansible crée un dossier dédié (par exemple /opt/docker_services/gitea/) sur la VM cible.
- Copie des fichiers docker-compose.yml : chaque fichier de composition est transféré depuis le dépôt vers le répertoire correspondant sur la VM.
- Déploiement des services : lancement de tous les conteneurs via docker compose (v2 minimum) pour chaque service.

Le dépôt contient un fichier principal deploy-all.yml qui sert de point d'entrée. Ce fichier référence et exécute tous les playbooks du dossier playbooks_ansible/. Chaque playbook se charge de déployer un service spécifique.

7.Vérification après le déploiement

Une fois le playbook terminé sans erreur, vérifier l'état de l'infrastructure depuis l'une des VMs :

docker ps.

V. PERFORMANCE, MONITORING ET TESTS DE CHARGE

A. Outils de monitoring mis en place

❖ Outils de monitoring utilisés

L'infrastructure de monitoring repose sur trois composants :

- **Prometheus** : collecteur de métriques temps réel qui interroge régulièrement les endpoints /metrics des services et exporters, et stocke les données en séries temporelles pour analyse et historique.
- **cAdvisor** : exporter Docker qui expose les métriques détaillées de chaque conteneur (CPU, mémoire, réseau, disque), que Prometheus récupère en permanence.
- **Node Exporter** : exporter système qui fournit les métriques de la VM hôte (CPU global, mémoire, disque, charge système).
- **Grafana** : plateforme de visualisation qui se connecte à Prometheus et crée des tableaux de bord interactifs affichant graphiques et indicateurs synthétisés des performances.

Cette pile légère permet une surveillance complète sans surcharger les ressources limitées de la VM.

❖ Présentation du tableau de bord :





Un tableau de bord Grafana nommé **"Monitor Docker"** a été configuré pour centraliser toutes les métriques de l'infrastructure. Il affiche en temps réel :

- **Vue d'ensemble globale** : trois indicateurs clés en haut du dashboard montrent le nombre de conteneurs en cours d'exécution (22 conteneurs), la mémoire totale utilisée (2,10 GiB) et l'utilisation CPU totale de la VM (5,89%).
- **Utilisation CPU par conteneur** : un graphique temporel qui trace la consommation CPU de chaque conteneur, permettant d'identifier rapidement les services gourmands ou les pics de charge.
- **Utilisation mémoire par conteneur** : un graphique empilé montrant la répartition de la RAM entre tous les conteneurs. On peut observer que Prometheus (344-370 MiB), Gitea (132 MiB + gitea_db 31.1 MiB), Trilium (121 MiB), cAdvisor (84 MiB) sont les principaux consommateurs de mémoire.
- **Trafic réseau (Rx/Tx)** : deux graphiques distincts affichent le débit entrant (Network Rx) et sortant (Network Tx) pour chaque conteneur. Les pics de trafic visibles correspondent aux échanges réseau des services (par exemple synchronisation Syncthing, requêtes HTTP vers Nextcloud/Grafana, collecte de métriques par Prometheus).

Ce tableau de bord unique permet de surveiller l'ensemble de l'infrastructure d'un seul coup d'œil et de diagnostiquer rapidement les problèmes de performance ou de ressources.

❖ **Configuration**

➢ **Conteneur Prometheus**

- Service prometheus basé sur l'image prom/prometheus:latest, connecté au réseau Docker monitoring avec redémarrage unless-stopped.
- Port interne 9090 exposé sur le port 8086 de la VM (8086:9090).

- Volumes : ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml pour charger la configuration de collecte, et prometheus-data:/prometheus pour la persistance des séries temporelles.
- Commandes spécifiques : --config.file pour indiquer la configuration, --storage.tsdb.path pour le répertoire de stockage, --storage.tsdb.retention.time=200h pour conserver 200 heures d'historique, et --web.enable-lifecycle pour recharger la configuration à chaud.

➢ **Conteneur Grafana**

- Service grafana basé sur l'image grafana/grafana:latest, dépendant du service prometheus et relié au réseau monitoring avec redémarrage unless-stopped.
- Port interne 3000 exposé sur le port 3000 de la VM (3000:3000).
- Volume grafana-data:/var/lib/grafana pour la persistance des dashboards, des sources de données et des configurations utilisateur.
- Variables d'environnement : GF_SECURITY_ADMIN_USER=admin, GF_SECURITY_ADMIN_PASSWORD=admin pour définir les identifiants administrateur par défaut, et GF_USERS_ALLOW_SIGN_UP=false pour bloquer les inscriptions publiques.

➢ **Conteneur Node Exporter**

- Service node-exporter basé sur l'image prom/node-exporter:latest, relié au réseau monitoring avec redémarrage unless-stopped.
- Port interne 9100 exposé sur le port 8084 de la VM (8084:9100).
- Volumes montés en lecture seule : /proc:/host/proc:ro, /sys:/host/sys:ro, /:/rootfs:ro pour accéder aux métriques du système hôte.
- Commandes spécifiques : --path.procfs, --path.rootfs, --path.sysfs pour indiquer les chemins des pseudo-systèmes de fichiers, et --collector.filesystem.mount-points-exclude pour exclure les montages système inutiles.

➢ **Conteneur cAdvisor**

- Service cadvisor basé sur l'image gcr.io/cadvisor/cadvisor:latest, relié au réseau monitoring avec redémarrage unless-stopped.
- Mode privileged: true et accès au périphérique /dev/kmsg pour accéder aux métriques noyau et conteneurs.
- Port interne 8080 exposé sur le port 8085 de la VM (8085:8080).
- Volumes montés en lecture seule : /:/rootfs:ro, /var/run:/var/run:ro, /sys:/sys:ro, /var/lib/docker:/var/lib/docker:ro, /cgroup:/cgroup:ro pour surveiller tous les conteneurs Docker et leurs ressources.

➢ **Configuration Prometheus (prometheus.yml)**

- Scrape interval global de 15 secondes pour interroger les métriques.

- Trois jobs définis : prometheus pour se monitorer lui-même (localhost:9090), node-exporter pour les métriques système (node-exporter:9100), et cadvisor pour les métriques conteneurs (cadvisor:8080).

Nous avons effectué une configuration Nginx aussi à ce niveau, mais seulement sur grafana.

Reverse proxy NGINX :

- Bloc server similaire à celui de Trilium, à l'écoute sur le port 80 avec authentification basique via auth_basic et fichier /etc/nginx/.htpasswd.
- Différence principale : le nom de domaine utilisé est grafana.tpdocker.istic.univ-rennes1.fr et le proxy redirige vers http://localhost:3000, correspondant au port HTTP publié par le conteneur Grafana.

B. Outil de load-testing

K6 a été choisi car il offre une excellente balance entre simplicité (scripts en JavaScript), performance et légèreté. Il génère une charge importante sans surcharger la VM (4 Go RAM). Par rapport à JMeter, trop lourd et complexe en conteneur, et à wrk, limité aux scénarios simples, K6 permet des tests complexes avec rampes de charge progressives, tout en restant très efficace en ressources.

❖ Configuration

Le contenu du docker n'étant pas long nous le mettons directement à disposition :

```
services:
k6:
image: grafana/k6:latest
container_name: k6
volumes:
- ./k6:/scripts
command: ["run", "/scripts/script.js"]
```

Ci-dessous un exmple de script :

```
import http from 'k6/http';

import { check, sleep } from 'k6';


export const options = {

 vus: 10,        // 10 utilisateurs virtuels

 duration: '30s',  // pendant 30 secondes

};

export default function () {
```

```
  const res = http.get('http://username:pasword@wiki.tpdocker.istic.univ-rennes1.fr/');
  check(res, {
    'status 200': (r) => r.status === 200,
  });
  sleep(1);
}
```

Au niveau du **http.get**, le nom de domaine est précédé de **username :password@** pour faire référence aux identifiants de connexion Nginx (s'il y en a). Ils sont à retirer dans le cas où le service n'en est pas doté.

C. Load Balancing

Le load balancing a été implémenté sur le service WordPress afin de distribuer la charge entre plusieurs instances et d'améliorer la disponibilité.

Le load balancing pour WordPress se fait en deux étapes : d'abord la duplication de l'application en deux conteneurs distincts, ensuite la répartition du trafic via Nginx.

- Définir un service MariaDB unique partagé par les deux instances WordPress, avec un volume persistant pour les données de la base.

- Partager aussi un volume pour les fichiers WordPress (wordpress_app_data) afin que les deux conteneurs servent exactement les mêmes contenus (thèmes, plugins, uploads).

Démarrage des deux backends

Lancer le docker-compose.yml dans le dossier WordPress :

```
cd esir-as-a-service-2025/wordpress
docker compose up -d
```

Vérifier que les deux conteneurs WordPress sont bien en écoute sur les ports hôtes :

```
docker ps | grep wordpress
```

Résultat attendu : deux lignes avec 0.0.0.0:8081->80 et 0.0.0.0:8087->80.

Si les réponses envoient un code HTTP 200 / 301 / 302, les backends sont prêts pour le load balancing.

Configuration du load balancing dans Nginx

1. Définition du bloc upstream

Sur la VM, dans le fichier de configuration dédié à WordPress (par exemple **/etc/nginx/sites-available/wordpress.conf)**

```
upstream wordpress_backend {
    least_conn;
    server 127.0.0.1:8081;
    server 127.0.0.1:8087;
}
```

La directive least_conn fait en sorte que Nginx envoie les nouvelles requêtes vers le serveur ayant le moins de connexions actives, ce qui répartit la charge de manière équilibrée.

2. Création du vhost WordPress (si n'existe pas deja)

Dans le même fichier, définir le serveur virtuel qui écoutera sur le port 80 et fera suivre le trafic vers le bloc upstream :

```
server {
  listen 80;
  server_name wordpress.${DOMAIN_BASE};

  auth_basic "Accès réservé";
  auth_basic_user_file /etc/nginx/.htpasswd;

  location / {
    proxy_pass http://wordpress_backend;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
  }
}
```

3. Activation du site et rechargement de Nginx

Créer le lien symbolique si nécessaire :

```
ln -s /etc/nginx/sites-available/wordpress.conf /etc/nginx/sites-enabled/wordpress.conf
```

Vérifier la configuration :

```
sudo nginx -t
```

Recharger Nginx :

```
sudo systemctl reload nginx
```

4. Vérifications et validation du load balancing

```
curl -I http://wordpress.${DOMAIN_BASE}
```

Si l'authentification est activée, entrer l'utilisateur/mot de passe défini dans /etc/nginx/.htpasswd.

- Actualiser plusieurs fois la page WordPress et, en parallèle, regarder les logs des deux conteneurs :

```
docker logs wordpress_wordpress1_1 -f

docker logs wordpress_wordpress2_1 -f
```

Les requêtes doivent apparaître de manière alternée (ou au moins répartie) dans les deux logs, preuve que Nginx distribue bien le trafic entre les deux backends.

CONCLUSION

Ce projet a permis de déployer une infrastructure micro-services complète et fonctionnelle sur l'infrastructure ISTIC, intégrant huit services critiques (prise de notes, synchronisation, gestion de code, messagerie, site web, photos, gestion de mots de passe, wiki), un monitoring en temps réel et une automatisation robuste via Ansible. L'architecture à deux VMs (une frontale NGINX, une applicative) offre une bonne séparation des responsabilités et limite la surface d'attaque, tandis que tous les services sont opérationnels, sécurisés par authentification et isolation réseau, et testés sous charge. L'ensemble confirme la viabilité d'une approche DevOps pragmatique, combinant containerisation, orchestration légère, monitoring et documentation rigoureuse, tout en ouvrant la voie à des améliorations futures (SSO centralisé, sauvegardes chiffrées, WAF).

La mise en place n'a toutefois pas été triviale et plusieurs difficultés ont façonné l'architecture finale. Certains services, comme Gitea, se comportaient mal derrière l'authentification basic de NGINX, ce qui nous a obligés à chercher d'autres moyens de sécuriser l'accès (par exemple en forçant la connexion applicative et en désactivant l'auth basic sur certains endpoints). De même, cAdvisor ne fonctionnait pas avec la version initiale de Docker fournie sur la VM : il a fallu réinstaller une version plus récente compatible, ce qui a rappelé l'importance de bien contrôler les versions de la chaîne Docker. Vaultwarden fonctionnait parfaitement en local mais restait bloqué en chargement une fois déployé ; le problème venait de l'absence de HTTPS. Enfin, la réservation des ports a également été source de confusion : certains ports répondaient sans problème alors que d'autres restaient injoignables sans qu'il s'agisse d'un pare-feu ou d'un conflit évident, ce qui a nécessité des tests.

Au final, ces obstacles ont été l'occasion de mieux comprendre les interactions entre reverse proxy, services applicatifs, réseau Docker et contraintes de versionning, et ont renforcé la solidité de l'architecture et de la démarche de déploiement automatisé.